CODEVAULT · PROGRAMMING PRACTICE LIBRARY

C Programming

100 Programs from Beginner to Advanced

A chapter-wise practice library covering input/output, operators, loops, arrays, strings, functions, pointers and file handling — for college practical exams, interview preparation and your GitHub portfolio.

C11 | GCC | VS Code | 100 Programs | 10 Chapters

AUTHOR · Sasidharan S.

PROJECT · CodeVault — Programming Practice Library

EDITION · 2026

Table of Contents

Preface 3

Chapter 1: Basic Input and Output 4

- Personalized Greeting . . . 4
- Student Information Card . . . 6
- Simple Bill Calculator . . . 10
- Marksheet Header Printer . . . 13
- Distance Converter (Kilometres to Miles) . . . 16
- Temperature Converter (Celsius to Fahrenheit) . . . 18
- Salary Slip Generator . . . 20
- Time Formatter (Minutes to Hours and Minutes) . . . 24
- Library Book Details Card . . . 26
- Personal Profile Card . . . 30

Chapter 2: Operators and Expressions 34

Chapter 3: Conditional Statements 35

Chapter 4: Looping Statements 36

Chapter 5: Pattern Programs 37

Chapter 6: Arrays 38

Chapter 7: Strings 39

Chapter 8: Functions and Recursion 40

Chapter 9: Pointers and Structures 41

Chapter 10: File Handling and Mini Projects 42

Conclusion 43

Preface

Welcome to the CodeVault C Programming collection. This book brings together one hundred carefully chosen C programs, organised into ten chapters of ten programs each. The chapters move step by step from the simplest possible input/output program up to four mini-projects that combine file handling, structures and pointers in the way a real application does.

Every program in this collection follows the same fourteen-part layout: number, title, difficulty, aim, concepts used, problem description, algorithm, source code, line-by-line explanation, sample input, expected output, output explanation, common errors with solutions, and one practice question. The expected output shown for every program is a real terminal capture, not a hand prediction — each program is compiled with gcc -std=c11 -Wall -Wextra and run with the sample input before its output is pasted into this book.

The collection is designed for college practical-exam preparation, for a first programming interview, and as the spine of a professional GitHub portfolio. Read the chapters in order, type every program into your editor, compile it, run it with the sample input, then attempt the practice question at the end. By the time you reach the mini-projects in Chapter 10, you will be writing C with confidence.

Author: Sasidharan S. · **Project:** CodeVault — Programming Practice Library · **Edition:** 2026

Chapter 1: Basic Input and Output

Basic input and output are the first things every C programmer learns. The programs in this chapter read values from the keyboard using scanf() and write results to the screen using printf(). They cover the four fundamental format specifiers — %s for strings, %d for integers, %f for floats and %c for characters — and show how to mix them in a single program to build small, real-world utilities such as a bill calculator, a salary slip, a temperature converter and a book catalogue card.

By the end of this chapter you will be able to read several inputs of different types in sequence, format the output with field widths and precision specifiers, and lay out a decorated card on the screen. These skills are the foundation for every later chapter — none of which will make sense if you cannot reliably read a value from the keyboard and print a value to the screen.

PROGRAM P01 · Beginner

Personalized Greeting

Aim. Read the user's name from the keyboard and display a personalised greeting message on the screen.

Concepts used.

- Preprocessor directive #include
- The main() function and the int return type
- Output with printf()
- String input with scanf() and the %s format specifier
- Standard header <stdio.h>

Problem description

A program should ask the user for their name and then greet them by name. This is the classic "first program" that introduces how C reads input from the keyboard and writes output to the screen. The program must read a single word (no spaces) and print it back inside a greeting sentence.

Algorithm

1. Include the standard input/output header <stdio.h>.
2. Declare a character array name[50] large enough to hold a name.
3. Display the prompt "Enter your name: " using printf().
4. Read a single word from the keyboard into name using scanf("%49s", name).
5. Print the personalised greeting using printf("Hello, %s! ...", name).
6. Return 0 to the operating system to indicate success.

Source code

```c
/*
 * CodeVault - C Programming Practice Library
 * Chapter 1: Basic Input and Output
 *
 * Program 01: Personalized Greeting
 * Difficulty: Beginner
 *
 * Aim:
 *   Read the user's name from the keyboard and display a personalised
 *   greeting message on the screen.
 *
 * Concepts used:
 *   - #include preprocessor directive
 *   - main() function and return type
 *   - printf() for output
 *   - scanf() for string input with %s format specifier
 *   - Standard header &lt;stdio.h&gt;
 *
 * Compilation (GCC, C11):
 *   gcc -std=c11 -Wall -o P01_Personalized_Greeting P01_Personalized_Greeting.c
 *
 * Run:
 *   ./P01_Personalized_Greeting
 */

#include &lt;stdio.h&gt;

int main(void)
{
    char name[50];   /* buffer to hold the user's name (max 49 chars + '\0') */

    /* Prompt the user to enter a name */
    printf("Enter your name: ");

    /* Read a single word (no spaces) from standard input into name[] */
    scanf("%49s", name);

    /* Print the personalised greeting */
    printf("Hello, %s! Welcome to C programming.\n", name);

    return 0;   /* indicate successful termination to the operating system */
}
```

Explanation of the code

The #include <stdio.h> line brings in the declarations of printf() and scanf(). The int main(void) function is where every C program begins. Inside main, char name[50] reserves a 50-character buffer to store the name. The printf("Enter your name: ") call writes the prompt to the screen without a trailing newline, so the user types on the same line. scanf("%49s", name) reads up to 49 characters from the keyboard and stores them in name (the 50th slot is reserved for the terminating \0). The format specifier %49s prevents buffer overflow if the user types a very long name. The final printf embeds the name into the greeting using %s, and return 0 signals a clean exit.

Sample input

keyboard

```
Sasidharan
```

Expected output (real terminal capture)

terminal

```
Enter your name: Hello, Sasidharan! Welcome to C programming.
```

Explanation of the output

The prompt "Enter your name: " is printed first, the user types Sasidharan and presses Enter. The program then prints the greeting sentence with the entered name substituted in place of %s. The output is on a single line because printf only places a newline where you explicitly write \n.

Common errors and solutions

Common Error	Solution
Forgetting the & in scanf("%s", &name).	Actually for a char array used as a string, you write name (no &) because the array name already decays into a pointer. Adding & would be wrong here. The & is required for scalar variables like int or float.
Misspelling printf as print or Printf.	C is case-sensitive. The function is printf all lowercase. Wrong spellings cause a compile-time "implicit declaration" warning or a link error.
Not allocating enough space in the name array.	Use at least char name[50] for a person's name and limit the read with %49s. A buffer that is too small overflows when the user types a long name.
Using scanf("%s", name) without a width limit.	Replace with scanf("%49s", name) so the user cannot overflow the buffer by typing more than 49 characters.

Practice question

> *Modify the program to ask the user for both their first name and last name (read as two separate words) and print a single greeting that uses both, e.g. Hello, Sasidharan S.!*

PROGRAM P02 · Beginner

Student Information Card

Aim. Read a student's name, age, roll number and branch from the keyboard and print a neatly formatted student information card.

Concepts used.

- Mixing %s, %d format specifiers in one program
- The address-of operator & for scalar variables
- Multi-line formatted output using escape sequences
- char arrays as string containers

Problem description

A college office automation system needs to capture a student's basic details and print them in a card format that can be pasted into a file. The program must read a name, an integer age, an integer roll number, and a

branch name, then display all four fields inside a bordered card.

Algorithm

1. Declare two char arrays for name and branch, and two int variables for age and roll number.
2. Prompt for and read each value with the correct format specifier.
3. Use & for the int variables; omit it for the arrays.
4. Print a top border of = characters.
5. Print the card title centred.
6. Print each field on its own line using printf with the appropriate specifier.
7. Print a bottom border matching the top.

Source code

```
/*
 * CodeVault - C Programming Practice Library
 * Chapter 1: Basic Input and Output
 *
 * Program 02: Student Information Card
 * Difficulty: Beginner
 *
 * Aim:
 *   Read a student's name, age, roll number and branch from the keyboard
 *   and print a neatly formatted student information card.
 *
 * Concepts used:
 *   - Multiple inputs of mixed data types in a single program
 *   - %s (string), %d (integer), and integer field-width specifiers
 *   - Formatted multi-line output using printf() escape sequences
 *   - char arrays as string containers
 *
 * Compilation (GCC, C11):
 *   gcc -std=c11 -Wall -o P02_Student_Information_Card P02_Student_Information_Card.c
 *
 * Run:
 *   ./P02_Student_Information_Card
 */

#include &lt;stdio.h&gt;

int main(void)
{
    char name[50];        /* student full name (single word) */
    char branch[30];      /* branch / department name       */
    int  age;             /* student age in years           */
    int  roll_number;     /* roll / registration number     */

    /* --- Read inputs one by one --- */
    printf("Enter student name: ");
    scanf("%49s", name);

    printf("Enter age: ");
    scanf("%d", &amp;age);

    printf("Enter roll number: ");
    scanf("%d", &amp;roll_number);

    printf("Enter branch: ");
    scanf("%29s", branch);

    /* --- Print the formatted information card --- */
    printf("\n");
    printf("========================================\n");
    printf("        STUDENT INFORMATION CARD        \n");
    printf("========================================\n");
    printf(" Name        : %s\n", name);
    printf(" Age         : %d years\n", age);
    printf(" Roll Number : %d\n", roll_number);
    printf(" Branch      : %s\n", branch);
    printf("========================================\n");

    return 0;
}
```

Explanation of the code

Two char arrays (name[50] and branch[30]) hold the string inputs, and two int variables (age and roll_number) hold the numeric inputs. Each printf shows a prompt before the matching scanf. Notice the & in scanf("%d", &age) — it is needed because age is a scalar integer, so we must pass its address. The array names name and branch are passed without & because an array name already evaluates to a pointer to its first element. The final

block of printf statements prints the card; the width specifiers (e.g. %d) simply substitute the values, and the literal text and = signs form the borders.

Sample input

keyboard

```
Sasidharan
20
45
CSE
```

Expected output (real terminal capture)

terminal

```
Enter student name: Enter age: Enter roll number: Enter branch:
========================================
        STUDENT INFORMATION CARD
========================================
 Name : Sasidharan
 Age : 20 years
 Roll Number : 45
 Branch : CSE
========================================
```

Explanation of the output

The five prompt lines appear in order (one per scanf). After the user has typed all four values, the program prints a blank line, the top border, the centred title, the four detail lines, and the bottom border. Each value is substituted at the position of its format specifier — %s for strings and %d for integers.

Common errors and solutions

Common Error	Solution
Using scanf("%d", age) without the &.	Always use &age for scalar variables. Without the &, scanf receives an arbitrary address (the value of age, not its location), leading to a crash or memory corruption.
Forgetting the field width on string reads.	Use %49s for a 50-element array. Without the width limit, an over-long input overflows the buffer.
Mixing up the order of inputs.	Each printf prompt must match the scanf that follows. If you prompt for "age" but call scanf for roll_number, the program still runs but stores the wrong value.
Printing %d with a string variable.	Use %s for char arrays and %d for integers. Mismatched specifiers produce garbage output or a crash.

Practice question

> *Add two more fields — email ID and phone number — and print them inside the same card. Remember to choose appropriate buffer sizes for the new strings.*

PROGRAM P03 · Beginner

Simple Bill Calculator

Aim. Read an item name, the quantity purchased and its unit price; compute the total amount and print a simple itemised bill.

Concepts used.

- Mixing int and float inputs in one program
- Arithmetic on input values (quantity * unit_price)
- Currency formatting with %.2f (two decimal places)
- Field-width specifiers for tabular output

Problem description

A small shopkeeper wants a program that takes the name of an item, the number of units bought, and the price of one unit, then prints a receipt showing the item, quantity, unit price and the total amount payable. Currency must always be shown with exactly two decimal places.

Algorithm

1. Declare a char array for the item name, an int for quantity, and two float variables for unit price and total.
2. Read the item name, quantity and unit price.
3. Compute total = quantity * unit_price.
4. Print a top border, the receipt title, and the item line.
5. Print the quantity, unit price and total — all formatted with %.2f for currency.
6. Print a bottom border to close the receipt.

Source code

```
/*
 * CodeVault - C Programming Practice Library
 * Chapter 1: Basic Input and Output
 *
 * Program 03: Simple Bill Calculator
 * Difficulty: Beginner
 *
 * Aim:
 *   Read the name of an item, the quantity purchased and its unit price.
 *   Compute the total amount and print a simple itemised bill.
 *
 * Concepts used:
 *   - Mixing int and float inputs in the same program
 *   - Arithmetic on input values (quantity * unit_price)
 *   - Currency formatting with %.2f (two decimal places)
 *   - Field-width specifiers for tabular output
 *
 * Compilation (GCC, C11):
 *   gcc -std=c11 -Wall -o P03_Bill_Calculator P03_Bill_Calculator.c
 *
 * Run:
 *   ./P03_Bill_Calculator
 */

#include &lt;stdio.h&gt;

int main(void)
{
    char  item[40];     /* item name (single word, e.g. Notebook) */
    int   quantity;     /* number of units purchased               */
    float unit_price;   /* price of one unit in rupees             */
    float total;        /* computed total amount                   */

    /* --- Read inputs --- */
    printf("Enter item name: ");
    scanf("%39s", item);

    printf("Enter quantity: ");
    scanf("%d", &amp;quantity);

    printf("Enter unit price (Rs): ");
    scanf("%f", &amp;unit_price);

    /* --- Compute the total bill --- */
    total = quantity * unit_price;

    /* --- Print the itemised bill --- */
    printf("\n");
    printf("------------------------------\n");
    printf("       SIMPLE BILL RECEIPT     \n");
    printf("------------------------------\n");
    printf(" Item        : %s\n", item);
    printf(" Quantity    : %d\n", quantity);
    printf(" Unit Price  : Rs. %.2f\n", unit_price);
    printf("------------------------------\n");
    printf(" TOTAL       : Rs. %.2f\n", total);
    printf("------------------------------\n");

    return 0;
}
```

Explanation of the code

The variables are declared with three different types to show how C handles mixed data: char item[40] for the name, int quantity for a whole-number count, and float unit_price, total for currency. The expression total = quantity * unit_price performs integer-to-float promotion automatically (the integer is converted to a float for the multiplication). The format specifier %.2f in printf rounds the float to two decimal places, which is the

standard for currency. The dashed borders are made from a literal string of - characters and the title is centred by hand with leading spaces.

Sample input

keyboard

```
Notebook
5
35.50
```

Expected output (real terminal capture)

terminal

```
Enter item name: Enter quantity: Enter unit price (Rs):
------------------------------
       SIMPLE BILL RECEIPT
------------------------------
 Item : Notebook
 Quantity : 5
 Unit Price : Rs. 35.50
------------------------------
 TOTAL : Rs. 177.50
------------------------------
```

Explanation of the output

The three prompts appear in order; after the user types "Notebook", 5, and 35.50, the program multiplies 5 by 35.50 to get 177.50 and prints the receipt. All currency values show two decimal places thanks to %.2f. The total is right-aligned under the dashed line because we used a fixed-width format inside the literal text.

Common errors and solutions

Common Error	Solution
Using %d to print a float.	Use %f or %.2f for floats. %d on a float produces garbage output because the bit patterns are interpreted differently.
Forgetting & on the float variables.	scanf("%f", &unit_price) is correct. Without &, scanf writes to a random address.
Computing the total before reading the inputs.	Order matters: read inputs first, then compute. Otherwise you multiply zero by zero.
Using integer division by mistake (e.g. 9/5).	Integer division truncates. Use 9.0f / 5.0f for floating-point division.

Practice question

> *Extend the program to apply a 5% discount on the total when the quantity is at least 10. Print the discount amount and the final payable amount on the bill.*

PROGRAM P04 · Beginner

Marksheet Header Printer

Aim. Read a student's name, class, roll number, subject and examination date, then print a neatly decorated marksheet header.

Concepts used.

- Multiple string inputs in sequence
- Reading an integer in between strings
- Decorative output using repeated characters and escape sequences
- A simple, alignment-safe box layout

Problem description

A school wants a quick way to print a uniform header at the top of every answer sheet. The program must read five fields — student name, class, roll number, subject and exam date — and print them inside a header made of two horizontal rules with a centred title in between. The header must always look aligned even when the inputs have different lengths.

Algorithm

1. Declare five variables: three char arrays, one int, one char array for the date.
2. Read each value with a matching prompt and the correct format specifier.
3. Print a row of = characters as the top border.
4. Print the centred title "EXAMINATION MARKSHEET".
5. Print a second row of = characters.
6. Print each field on its own line, with the label fixed-width and the value left-aligned.
7. Print a final row of = characters as the bottom border.

Source code

```c
/*
 * CodeVault - C Programming Practice Library
 * Chapter 1: Basic Input and Output
 *
 * Program 04: Marksheet Header Printer
 * Difficulty: Beginner
 *
 * Aim:
 *   Read a student's name, class, roll number, subject and the examination
 *   date, then print a neatly decorated marksheet header suitable for the
 *   top of an answer sheet or report card.
 *
 * Concepts used:
 *   - Multiple string inputs in sequence
 *   - Reading an integer in between strings
 *   - Decorative output using repeated characters and escape sequences
 *   - Field-width specifiers for alignment
 *
 * Compilation (GCC, C11):
 *   gcc -std=c11 -Wall -o P04_Marksheet_Header P04_Marksheet_Header.c
 *
 * Run:
 *   ./P04_Marksheet_Header
 */

#include &lt;stdio.h&gt;

int main(void)
{
    char student_name[50];
    char class_name[20];
    char subject[30];
    char exam_date[15];     /* dd/mm/yyyy as a single token */
    int  roll_number;

    /* --- Read inputs --- */
    printf("Enter student name : ");
    scanf("%49s", student_name);

    printf("Enter class        : ");
    scanf("%19s", class_name);

    printf("Enter roll number  : ");
    scanf("%d", &amp;roll_number);

    printf("Enter subject      : ");
    scanf("%29s", subject);

    printf("Enter exam date    : ");
    scanf("%14s", exam_date);

    /* --- Print the decorated marksheet header ---
     * The box uses 44 '=' characters as borders, with the data printed
     * in between using a simple, alignment-safe format.
     */
    printf("\n");
    printf("============================================\n");
    printf("         EXAMINATION MARKSHEET              \n");
    printf("============================================\n");
    printf(" Name    : %s\n", student_name);
    printf(" Class   : %s\n", class_name);
    printf(" Roll No.: %d\n", roll_number);
    printf(" Subject : %s\n", subject);
    printf(" Date    : %s\n", exam_date);
    printf("============================================\n");

    return 0;
}
```

Explanation of the code

The five declarations cover three string inputs (student_name, class_name, subject, exam_date) and one integer (roll_number). Each scanf uses a width-limited format specifier to prevent buffer overflow. The output section uses a 44-character border of = signs. The title is centred manually using leading spaces inside the literal string. Each data line uses a fixed label (Name, Class, etc.) followed by a colon and the value, which keeps the labels aligned regardless of the value length. This "no closing pipe" design avoids the alignment bugs you get when you try to close a box on the right edge.

Sample input

keyboard

```
Rahul
10A
42
Mathematics
15/11/2026
```

Expected output (real terminal capture)

terminal

```
Enter student name : Enter class : Enter roll number : Enter subject : Enter exam date :
============================================
         EXAMINATION MARKSHEET
============================================
 Name : Rahul
 Class : 10A
 Roll No.: 42
 Subject : Mathematics
 Date : 15/11/2026
============================================
```

Explanation of the output

The five prompts are printed one after another; after the user has entered all five values, the program prints a blank line, then the bordered header. The title line is followed by the five detail lines, each starting with a fixed-width label, then a final border closes the header. Because the labels are fixed width, the values all start in the same column even when the names have different lengths.

Common errors and solutions

Common Error	Solution
Trying to align a closing | at the right edge of a box.	If your field widths do not add up exactly, the closing | drifts. Either drop the closing | (as this program does) or compute the exact padding for each row.
Reading strings with scanf("%s") and no width limit.	Always write %49s (or whatever size your buffer is) to prevent overflow.
Forgetting to put a space between the prompt and the user's typing position.	Always end the prompt with a space (e.g. "Enter name: ") so the typed value does not visually collide with the prompt text.
Using %s to read a date with slashes (15/11/2026).	%s stops at the first whitespace, but / is not whitespace, so the date is read correctly as a single token.

Practice question

> *Add a "Maximum Marks" and "Marks Obtained" field to the header. Both are integers. Print them on the same line, separated by a slash, e.g. Marks : 100 / 87.*

PROGRAM P05 · Beginner

Distance Converter (Kilometres to Miles)

Aim. Read a distance in kilometres from the user and print the equivalent distance in miles using the conversion factor 1 km = 0.621371 miles.

Concepts used.

- Floating-point input with %f
- Named constant via #define for the conversion factor
- Floating-point arithmetic
- Precision specifier %.3f for three decimal places

Problem description

A travel-planning tool needs a quick km-to-miles converter. The user types a distance in kilometres and the program prints the same distance expressed in miles, with three decimal places so small differences (e.g. between 1.0 km and 1.5 km) are visible.

Algorithm

1. Define the conversion factor KM_TO_MILES as 0.621371f using #define.
2. Declare two float variables: kilometres and miles.
3. Read the distance in kilometres with scanf("%f", &kilometres).
4. Compute miles = kilometres * KM_TO_MILES.
5. Print both values using %.3f so the output shows three decimal places.

Source code

```
/*
 * CodeVault - C Programming Practice Library
 * Chapter 1: Basic Input and Output
 *
 * Program 05: Distance Unit Converter (Kilometres -&gt; Miles)
 * Difficulty: Beginner
 *
 * Aim:
 *   Read a distance in kilometres from the user and print the equivalent
 *   distance in miles using the conversion factor 1 km = 0.621371 miles.
 *
 * Concepts used:
 *   - Floating-point input with %f
 *   - Using a named constant (#define) for the conversion factor
 *   - Floating-point arithmetic
 *   - Precision specifier %.3f to display three decimal places
 *
 * Compilation (GCC, C11):
 *   gcc -std=c11 -Wall -o P05_Distance_Converter P05_Distance_Converter.c
 *
 * Run:
 *   ./P05_Distance_Converter
 */

#include &lt;stdio.h&gt;

#define KM_TO_MILES  0.621371f   /* conversion factor: 1 km = 0.621371 miles */

int main(void)
{
    float kilometres;
    float miles;

    /* --- Read the distance in kilometres --- */
    printf("Enter distance in kilometres: ");
    scanf("%f", &amp;kilometres);

    /* --- Convert to miles --- */
    miles = kilometres * KM_TO_MILES;

    /* --- Display the result with three decimal places --- */
    printf("%.3f km = %.3f miles\n", kilometres, miles);

    return 0;
}
```

Explanation of the code

The #define KM_TO_MILES 0.621371f line creates a named constant — a symbolic name for the conversion factor. Using a name instead of a magic number makes the code easier to read and change. The f suffix tells the compiler the literal is a float (without it, the literal would be a double, which then gets implicitly narrowed). The expression kilometres * KM_TO_MILES multiplies the input by the conversion factor; because both operands are float, the result is float. The %.3f format specifier rounds the result to three decimal places — useful for distances where small differences matter.

Sample input

keyboard
12.5

Expected output (real terminal capture)

terminal

```
Enter distance in kilometres: 12.500 km = 7.767 miles
```

Explanation of the output

The user types 12.5 km. The program multiplies 12.5 by 0.621371 to get 7.7671375, which %.3f rounds to 7.767. Both numbers are printed with three decimal places, so the user sees 12.500 km = 7.767 miles.

Common errors and solutions

Common Error	Solution
Hardcoding the conversion factor inline.	Use #define. If the factor changes (e.g. a more precise value), you only edit one place.
Forgetting the & in scanf("%f", &kilometres).	Without &, scanf writes to a random address — usually a crash.
Using %d instead of %f for a float input.	%d reads an integer; %f reads a float. Mismatching these gives wrong values without a warning.
Forgetting the f suffix on the literal.	Without f, 0.621371 is a double. The multiplication still works but the result is implicitly narrowed back to float. Adding f makes the intent explicit and avoids the implicit conversion.

Practice question

> *Add a second conversion in the same program — miles to kilometres — and let the user choose the direction by typing 1 for km→mi or 2 for mi→km before entering the distance. (You will need a basic if; that is fine even though it appears in Chapter 3.)*

PROGRAM P06 · Beginner

Temperature Converter (Celsius to Fahrenheit)

Aim. Read a temperature in degrees Celsius from the user and convert it to degrees Fahrenheit using the standard formula F = C × 9/5 + 32.

Concepts used.

- Floating-point input and output
- Arithmetic expression with operator precedence
- Why 9.0f / 5.0f must be used instead of 9 / 5
- %.2f precision for clean two-decimal output

Problem description

A weather app needs a Celsius-to-Fahrenheit converter. The user types a temperature in Celsius and the program prints the Fahrenheit equivalent, rounded to two decimal places. The formula is F = C × 9/5 + 32.

Algorithm

1. Declare two float variables: celsius and fahrenheit.
2. Read the Celsius value with scanf("%f", &celsius).
3. Compute fahrenheit = (celsius * 9.0f / 5.0f) + 32.0f.
4. Print both values with %.2f for two decimal places.

Source code

```
/*
 * CodeVault - C Programming Practice Library
 * Chapter 1: Basic Input and Output
 *
 * Program 06: Temperature Converter (Celsius -&gt; Fahrenheit)
 * Difficulty: Beginner
 *
 * Aim:
 *   Read a temperature in degrees Celsius from the user and convert it to
 *   degrees Fahrenheit using the standard formula  F = C * 9/5 + 32.
 *
 * Concepts used:
 *   - Floating-point input and output
 *   - Arithmetic expression with operator precedence
 *   - The degree symbol is written using the literal UTF-8 character
 *     so the output renders correctly in modern terminals (Windows
 *     Terminal, GNOME Terminal, VS Code integrated terminal, etc.).
 *     If your terminal shows a strange glyph instead of the degree sign,
 *     simply delete the character and replace it with the letters "deg".
 *   - %.2f precision for clean two-decimal output
 *
 * Compilation (GCC, C11):
 *   gcc -std=c11 -Wall -o P06_Temperature_Converter P06_Temperature_Converter.c
 *
 * Run:
 *   ./P06_Temperature_Converter
 */

#include &lt;stdio.h&gt;

int main(void)
{
    float celsius;
    float fahrenheit;

    /* --- Read temperature in Celsius --- */
    printf("Enter temperature in Celsius: ");
    scanf("%f", &amp;celsius);

    /* --- Apply the conversion formula F = C * 9/5 + 32 ---
     * Note: 9.0f / 5.0f forces floating-point division. Using 9 / 5
     * would yield integer division (= 1) and give the wrong answer.
     */
    fahrenheit = (celsius * 9.0f / 5.0f) + 32.0f;

    /* --- Display the result with two decimal places --- */
    printf("%.2f°C = %.2f°F\n", celsius, fahrenheit);

    return 0;
}
```

Explanation of the code

The expression (celsius * 9.0f / 5.0f) + 32.0f implements the formula F = C × 9/5 + 32. The parentheses are not strictly needed (multiplication and division have higher precedence than addition) but they make the formula explicit and easy to read. The .0f suffixes are critical: 9 / 5 in C is integer division and equals 1, not 1.8. Writing 9.0f / 5.0f forces floating-point division and gives the correct 1.8. The %.2f format specifier

rounds the output to two decimal places, which is the convention for weather temperatures.

Sample input

keyboard

```
37.5
```

Expected output (real terminal capture)

terminal

```
Enter temperature in Celsius: 37.50°C = 99.50°F
```

Explanation of the output

The user types 37.5. The program computes (37.5 × 9.0 / 5.0) + 32.0 = 67.5 + 32.0 = 99.5. With %.2f formatting, the output is 37.50°C = 99.50°F. The degree sign is the literal UTF-8 character ° embedded directly in the format string.

Common errors and solutions

Common Error	Solution
Writing 9 / 5 instead of 9.0f / 5.0f.	Integer division truncates, so 9 / 5 = 1 and the formula gives the wrong answer. Always use floating-point literals when you want floating-point division.
Forgetting the & in scanf("%f", &celsius).	Without &, scanf writes to a random address. Always use & for scalar variables.
Mixing up the order of operations.	The formula F = C × 9 / 5 + 32 works because multiplication and division happen before addition. If you want to be explicit, write (C × 9 / 5) + 32.
Using the wrong sign for negative inputs.	A negative Celsius value (e.g. -10) is correctly converted by the formula. Just make sure your scanf reads the minus sign — %f handles it.

Practice question

> *Make the program convert in both directions. Read a character d (for "to Fahrenheit") or c (for "to Celsius") before the temperature, then apply the correct formula.*

PROGRAM P07 · Beginner

Salary Slip Generator

Aim. Read an employee's name and basic salary; compute House Rent Allowance (HRA = 20% of basic), Dearness Allowance (DA = 10% of basic), and print a formatted salary slip showing basic, HRA, DA and gross salary.

Concepts used.

- Floating-point input with %f and string input with %s
- Percentage calculations using multiplication
- Named constants via #define for allowance rates
- Currency formatting with %.2f and field-width specifiers (%10.2f)

Problem description

A small company wants a simple salary-slip printer. The clerk types an employee's name and basic salary; the program computes the allowances (HRA at 20% of basic, DA at 10% of basic), adds them to the basic to get the gross, and prints a neat salary slip with all four amounts right-aligned in a column.

Algorithm

1. Define HRA_RATE as 0.20f and DA_RATE as 0.10f using #define.
2. Declare a char array for the employee name and four float variables for basic, HRA, DA and gross.
3. Read the employee name and basic salary.
4. Compute hra = basic * HRA_RATE and da = basic * DA_RATE.
5. Compute gross = basic + hra + da.
6. Print the slip: a top border, the title, the employee name, a separator, the four amounts right-aligned with %10.2f, a separator, and a bottom border.

Source code

```c
/*
 * CodeVault - C Programming Practice Library
 * Chapter 1: Basic Input and Output
 *
 * Program 07: Salary Slip Generator
 * Difficulty: Beginner
 *
 * Aim:
 *   Read an employee's name and basic salary. Compute House Rent Allowance
 *   (HRA = 20% of basic), Dearness Allowance (DA = 10% of basic), and print
 *   a formatted salary slip showing basic, HRA, DA and gross salary.
 *
 * Concepts used:
 *   - Floating-point input with %f and string input with %s
 *   - Percentage calculations using multiplication
 *   - Named constants (#define) for allowance rates
 *   - Currency formatting with %.2f
 *   - Multi-line formatted output
 *
 * Compilation (GCC, C11):
 *   gcc -std=c11 -Wall -o P07_Salary_Slip P07_Salary_Slip.c
 *
 * Run:
 *   ./P07_Salary_Slip
 */

#include &lt;stdio.h&gt;

#define HRA_RATE  0.20f   /* House Rent Allowance = 20% of basic  */
#define DA_RATE   0.10f   /* Dearness Allowance    = 10% of basic  */

int main(void)
{
    char  employee_name[50];
    float basic;
    float hra;
    float da;
    float gross;

    /* --- Read inputs --- */
    printf("Enter employee name: ");
    scanf("%49s", employee_name);

    printf("Enter basic salary (Rs): ");
    scanf("%f", &amp;basic);

    /* --- Compute allowances and gross salary --- */
    hra   = basic * HRA_RATE;
    da    = basic * DA_RATE;
    gross = basic + hra + da;

    /* --- Print the salary slip --- */
    printf("\n");
    printf("============================================\n");
    printf("            MONTHLY SALARY SLIP             \n");
    printf("============================================\n");
    printf(" Employee : %s\n", employee_name);
    printf("--------------------------------------------\n");
    printf(" Basic Salary : Rs. %10.2f\n", basic);
    printf(" HRA (20%%)    : Rs. %10.2f\n", hra);
    printf(" DA  (10%%)    : Rs. %10.2f\n", da);
    printf("--------------------------------------------\n");
    printf(" Gross Salary : Rs. %10.2f\n", gross);
    printf("============================================\n");

    return 0;
}
```

Explanation of the code

The two #define lines create named constants for the allowance rates. Using names instead of magic numbers makes the code easy to update if the rates change. The expression basic * HRA_RATE computes 20% of basic. The gross is the sum of the three components. The format specifier %10.2f in the printf calls right-aligns the currency value in a 10-character field with two decimal places, so all four amounts line up under each other in the slip.

Sample input

keyboard

```
Anitha
25000
```

Expected output (real terminal capture)

terminal

```
Enter employee name: Enter basic salary (Rs):
============================================
            MONTHLY SALARY SLIP
============================================
 Employee : Anitha
--------------------------------------------
 Basic Salary : Rs. 25000.00
 HRA (20%) : Rs. 5000.00
 DA (10%) : Rs. 2500.00
--------------------------------------------
 Gross Salary : Rs. 32500.00
============================================
```

Explanation of the output

The user types Anitha and 25000. The program computes HRA = 25000 × 0.20 = 5000, DA = 25000 × 0.10 = 2500, and gross = 25000 + 5000 + 2500 = 32500. The slip prints each amount right-aligned in a 10-character field, so the decimal points of all four amounts line up vertically. The %% in the printf string is the way to print a literal % character.

Common errors and solutions

Common Error	Solution
Writing 20% as the rate instead of 0.20f.	C has no percentage operator for literals. Write the rate as a decimal fraction and multiply.
Using %f instead of %.2f for currency.	%f prints six decimal places by default. %.2f gives the standard two-decimal currency format.
Forgetting the %% escape for a literal percent sign.	Inside a printf format string, % starts a conversion. To print a literal %, write %%.
Adding the basic twice when computing gross.	gross = basic + hra + da — make sure you do not accidentally write gross = basic + basic + hra + da.

Practice question

Add a tax deduction of 5% on the gross salary. Print a "Tax" line and a "Net Salary" line (gross minus tax) on the slip, both right-aligned with %10.2f.

PROGRAM P08 · Beginner

Time Formatter (Minutes to Hours and Minutes)

Aim. Read a duration in total minutes from the user and display it in the human-readable form "X hours Y minutes" using integer division and the modulo operator.

Concepts used.

- Integer input with %d
- Integer division / to obtain the hours part
- The modulo operator % to obtain the remaining minutes
- Formatted integer output with %d

Problem description

A bus-booking system stores trip durations in minutes but displays them in hours-and-minutes form. The user enters a number of minutes (for example 135) and the program prints the same duration as "2 hours 15 minutes".

Algorithm

1. Declare three int variables: total_minutes, hours, minutes.
2. Read the duration with scanf("%d", &total_minutes).
3. Compute hours = total_minutes / 60 (integer division gives the whole-hour part).
4. Compute minutes = total_minutes % 60 (the remainder after the hours are removed).
5. Print the result in the form X minutes = Y hours Z minutes.

Source code

```
/*
 * CodeVault - C Programming Practice Library
 * Chapter 1: Basic Input and Output
 *
 * Program 08: Time Formatter (Minutes -&gt; Hours and Minutes)
 * Difficulty: Beginner
 *
 * Aim:
 *   Read a duration in total minutes from the user and display it in the
 *   human-readable form "X hours Y minutes" using integer division (/)
 *   and the modulo operator (%).
 *
 * Concepts used:
 *   - Integer input with %d
 *   - Integer division (/) to obtain the hours part
 *   - Modulo operator (%) to obtain the remaining minutes
 *   - Formatted integer output with %d
 *
 * Compilation (GCC, C11):
 *   gcc -std=c11 -Wall -o P08_Time_Formatter P08_Time_Formatter.c
 *
 * Run:
 *   ./P08_Time_Formatter
 */

#include &lt;stdio.h&gt;

int main(void)
{
    int total_minutes;   /* input  : duration in minutes            */
    int hours;           /* output : whole-hour part               */
    int minutes;         /* output : remaining minutes after hours */

    /* --- Read the duration in minutes --- */
    printf("Enter total minutes: ");
    scanf("%d", &amp;total_minutes);

    /* --- Break the duration into hours and minutes --- */
    hours   = total_minutes / 60;
    minutes = total_minutes % 60;

    /* --- Display the formatted duration --- */
    printf("%d minutes = %d hours %d minutes\n",
           total_minutes, hours, minutes);

    return 0;
}
```

Explanation of the code

The two key operators are / (integer division) and % (modulo, or remainder). For an input of 135: 135 / 60 is 2 because integer division truncates the fractional part; 135 % 60 is 15 because 60 × 2 = 120, and 135 − 120 = 15. Together, the two operators split the duration into the whole-hour part and the leftover minutes. All three variables are int because minutes and hours are always whole numbers.

Sample input

keyboard

```
135
```

Expected output (real terminal capture)

terminal

```
Enter total minutes: 135 minutes = 2 hours 15 minutes
```

Explanation of the output

The user types 135. The program computes 135 / 60 = 2 and 135 % 60 = 15, so the printed line is 135 minutes = 2 hours 15 minutes. If the user types 60, the output is 60 minutes = 1 hours 0 minutes (grammar is left as a Chapter 3 exercise using a conditional).

Common errors and solutions

Common Error	Solution
Using total_minutes / 60.0 instead of total_minutes / 60.	Floating-point division gives 2.25 — wrong for counting whole hours. Use integer division with int operands.
Forgetting the & in scanf("%d", &total_minutes).	Always use & for scalar variables. Without it, scanf writes to a random address.
Mixing up % in scanf and % as the modulo operator.	Inside scanf and printf format strings, %d is a conversion specifier. As an arithmetic operator (e.g. 135 % 60), % is the modulo. The compiler tells them apart by context.
Printing the result with %f for integer variables.	Use %d for int variables. %f on an int gives garbage output.

Practice question

> *Extend the program to also break the hours into days and hours (1 day = 24 hours). For an input of 1500 minutes, the output should look like 1500 minutes = 1 days 1 hours 0 minutes.*

PROGRAM P09 · Beginner

Library Book Details Card

Aim. Read the details of a library book — title, author, ISBN, price and number of pages — and print a neatly formatted book information card suitable for a library catalogue entry.

Concepts used.

- Mixed data-type inputs: %s (string), %d (integer), %f (float)
- Field-width specifiers for left-aligned columns
- Currency formatting with %.2f
- Multi-line decorated output

Problem description

A library needs a quick way to print catalogue cards. The librarian types the book title, author, ISBN (as a string of digits), price and page count; the program prints a card with all five fields, formatted in a uniform layout so the cards can be filed together.

Algorithm

1. Declare three char arrays for title, author and ISBN, one float for price, and one int for pages.
2. Read each value with a matching prompt and the correct format specifier.
3. Print a top border of = characters.
4. Print the centred card title "LIBRARY BOOK INFORMATION".
5. Print each of the five fields on its own line with a fixed-width label.
6. Print a bottom border to close the card.

Source code

```
/*
 * CodeVault - C Programming Practice Library
 * Chapter 1: Basic Input and Output
 *
 * Program 09: Library Book Details Card
 * Difficulty: Beginner
 *
 * Aim:
 *   Read the details of a library book - title, author, ISBN, price and
 *   number of pages - and print a neatly formatted book information card
 *   suitable for a library catalogue entry.
 *
 * Concepts used:
 *   - Mixed data-type inputs: %s (string), %d (integer), %f (float)
 *   - Field-width specifiers for left-aligned columns (%-25s)
 *   - Currency formatting with %.2f
 *   - Multi-line decorated output
 *
 * Compilation (GCC, C11):
 *   gcc -std=c11 -Wall -o P09_Book_Details P09_Book_Details.c
 *
 * Run:
 *   ./P09_Book_Details
 */

#include &lt;stdio.h&gt;

int main(void)
{
    char  title[60];     /* book title            */
    char  author[40];    /* author name           */
    char  isbn[15];      /* ISBN as a string token */
    float price;         /* book price in rupees  */
    int   pages;         /* number of pages       */

    /* --- Read inputs --- */
    printf("Enter book title  : ");
    scanf("%59s", title);

    printf("Enter author name : ");
    scanf("%39s", author);

    printf("Enter ISBN        : ");
    scanf("%14s", isbn);

    printf("Enter price (Rs)  : ");
    scanf("%f", &amp;price);

    printf("Enter number of pages: ");
    scanf("%d", &amp;pages);

    /* --- Print the formatted book information card --- */
    printf("\n");
    printf("============================================\n");
    printf("          LIBRARY BOOK INFORMATION          \n");
    printf("============================================\n");
    printf(" Title  : %s\n", title);
    printf(" Author : %s\n", author);
    printf(" ISBN   : %s\n", isbn);
    printf(" Price  : Rs. %.2f\n", price);
    printf(" Pages  : %d\n", pages);
    printf("============================================\n");

    return 0;
}
```

Explanation of the code

The five declarations cover three strings, one float and one integer. The ISBN is stored as a string (char isbn[15]) because ISBNs can be very long and may include leading zeros (an int would drop the zeros). The price is a float so it can hold decimal values; %.2f formats it as currency. The pages are an int because they are always a whole number. The card uses the same border-and-label layout as P04, which keeps the labels aligned across books even when titles and authors have very different lengths.

Sample input

keyboard

```
LetUsC
YashavantKanetkar
9788183331630
450.00
658
```

Expected output (real terminal capture)

terminal

```
Enter book title : Enter author name : Enter ISBN : Enter price (Rs) : Enter number of pages:
==========================================
          LIBRARY BOOK INFORMATION
==========================================
 Title : LetUsC
 Author : YashavantKanetkar
 ISBN : 9788183331630
 Price : Rs. 450.00
 Pages : 658
==========================================
```

Explanation of the output

After the user types all five values, the program prints a blank line, then the bordered card with the centred title and the five fields. The price is shown as Rs. 450.00 thanks to %.2f, and the page count is shown as the integer it is.

Common errors and solutions

Common Error	Solution
Storing the ISBN as an integer.	ISBNs can have leading zeros (e.g. 978-0-13-110362-7) and may include dashes. Store them as strings, not as numbers.
Using %d to print a float price.	Always use %f or %.2f for floats. %d on a float produces garbage output.
Forgetting & on scanf("%f", &price).	Always use & for scalar variables (int, float, double, char). Without it, scanf writes to a random address.
Mixing up the buffer sizes and the width-limited reads.	For char title[60], write %59s. The width must be at most one less than the array size to leave room for the terminating \0.

Practice question

Add a "Year of Publication" field (integer) and an "Available Copies" field (integer). Print both on the same line, separated by a vertical bar, e.g. Year: 2021 | Copies: 5.

PROGRAM P10 · Beginner

Personal Profile Card

Aim. Read a person's name, age, height (in centimetres), blood group and city, then print a neatly formatted personal profile card mixing string, integer and floating-point data types.

Concepts used.

- Mixed data-type inputs: %s (string), %d (integer), %f (float)
- Field-width specifiers (%-20s, %6.2f) for aligned output
- Multi-line decorated output
- Reading several strings and numeric values in a single program

Problem description

A conference registration desk wants a quick profile printer. The attendee types their name, age, height, blood group and city; the program prints a profile card with all five fields, with the values aligned in a clean column so the cards look uniform across attendees.

Algorithm

1. Declare three char arrays for name, blood group and city, one int for age, and one float for height.
2. Read each value with a matching prompt and the correct format specifier.
3. Print a top border of = characters.
4. Print the centred card title "PERSONAL PROFILE CARD".
5. Print each field on its own line, using %-30s for the name (left-aligned in 30 chars), %d for age, %6.2f for height, and %s for blood group and city.
6. Print a bottom border to close the card.

Source code

```
/*
 * CodeVault - C Programming Practice Library
 * Chapter 1: Basic Input and Output
 *
 * Program 10: Personal Profile Card
 * Difficulty: Beginner
 *
 * Aim:
 *   Read a person's name, age, height (in centimetres), blood group and city,
 *   then print a neatly formatted personal profile card mixing string,
 *   integer and floating-point data types.
 *
 * Concepts used:
 *   - Mixed data-type inputs: %s (string), %d (integer), %f (float)
 *   - Field-width specifiers (%-20s, %6.2f) for aligned output
 *   - Multi-line decorated output
 *   - Reading several strings and numeric values in a single program
 *
 * Compilation (GCC, C11):
 *   gcc -std=c11 -Wall -o P10_Profile_Card P10_Profile_Card.c
 *
 * Run:
 *   ./P10_Profile_Card
 */

#include &lt;stdio.h&gt;

int main(void)
{
    char  name[50];       /* person's full name (single word) */
    char  blood_group[5]; /* e.g. A+, O-, B+                  */
    char  city[30];       /* city of residence                */
    int   age;            /* age in years                      */
    float height_cm;      /* height in centimetres            */

    /* --- Read inputs --- */
    printf("Enter your name        : ");
    scanf("%49s", name);

    printf("Enter your age         : ");
    scanf("%d", &amp;age);

    printf("Enter height in cm     : ");
    scanf("%f", &amp;height_cm);

    printf("Enter blood group      : ");
    scanf("%4s", blood_group);

    printf("Enter your city        : ");
    scanf("%29s", city);

    /* --- Print the formatted profile card --- */
    printf("\n");
    printf("==============================================\n");
    printf("           PERSONAL PROFILE CARD            \n");
    printf("==============================================\n");
    printf(" Name        : %-30s\n", name);
    printf(" Age         : %d years\n", age);
    printf(" Height      : %6.2f cm\n", height_cm);
    printf(" Blood Group : %s\n", blood_group);
    printf(" City        : %s\n", city);
    printf("==============================================\n");

    return 0;
}
```

Explanation of the code

The five declarations mix three string arrays, one integer and one float — the same mix you would see in any real-world form. The format specifier %-30s left-aligns the name in a 30-character field (the - forces left alignment; without it, the value would be right-aligned). The %6.2f for height means "total width 6, two decimal places", so 172.5 prints as 172.50. The %s specifiers for blood group and city just print the strings as they are. This program is the chapter's capstone: it pulls together strings, integers, floats, field widths and a clean box layout in a single piece of code.

Sample input

keyboard

```
Deepak
21
172.5
O+
Chennai
```

Expected output (real terminal capture)

terminal

```
Enter your name : Enter your age : Enter height in cm : Enter blood group : Enter your city :
==========================================
           PERSONAL PROFILE CARD
==========================================
 Name : Deepak
 Age : 21 years
 Height : 172.50 cm
 Blood Group : O+
 City : Chennai
==========================================
```

Explanation of the output

After the user types Deepak, 21, 172.5, O+, Chennai, the program prints the profile card. The name is left-aligned in a 30-char field (you can see trailing spaces after "Deepak"). The height is shown as 172.50 cm thanks to %6.2f. The card is closed by the same border that opened it.

Common errors and solutions

Common Error	Solution
Forgetting the - in %-30s.	Without the -, the value is right-aligned, which looks odd for names and cities. Use %-30s for left alignment.
Using %d for the height (a float).	Use %f or %6.2f for floats. %d on a float gives garbage output.
Reading the blood group with %s and no width limit.	Always write %4s for a 5-element array (e.g. char blood[5]) to prevent overflow.
Misaligning the labels.	Keep labels at a fixed width (e.g. Name : with the same number of spaces every time) so the values all start in the same column.

Practice question

Add two more fields — phone number (string) and email (string) — and print them at the bottom of the card. Make sure the labels stay aligned with the existing ones.

Chapter 2: Operators and Expressions

Status: Coming in the next release. **Number of programs:** 10

Planned topics

01. Arithmetic operators (+, -, *, /, %)

02. Relational operators (<, >, <=, >=, ==, !=)

03. Logical operators (&&, ||, !)

04. Bitwise operators (&, |, ^, ~, <<, >>)

05. Assignment operators (=, +=, -=, *=, /=, %=)

06. Increment and decrement (++ and --)

07. Conditional (ternary) operator (?:)

08. Sizeof and comma operator

09. Operator precedence and associativity

10. Type casting and mixed-mode arithmetic

Chapter notes

This chapter deepens the arithmetic introduced in Chapter 1. The 10 programs will exercise every C operator and show the difference between integer and floating-point division, the side effects of ++ and --, and how precedence changes the result of an expression.

Chapter 3: Conditional Statements

Status: Coming in the next release. **Number of programs:** 10

Planned topics

01. Simple if statement

02. if-else statement

03. Nested if-else

04. else-if ladder (multi-way decision)

05. switch statement (with and without break)

06. Checking even/odd, leap year, vowel/consonant

07. Finding the largest of three numbers

08. Simple calculator using switch

09. Grade calculator using else-if ladder

10. Validating user input

Chapter notes

This chapter introduces decision-making. The 10 programs will cover all five forms of conditional branching in C and contrast the if-else ladder with the switch statement, showing when each is the better tool.

Chapter 4: Looping Statements

Status: Coming in the next release. **Number of programs:** 10

Planned topics

01. while loop (entry-controlled)

02. do-while loop (exit-controlled)

03. for loop (count-controlled)

04. Nested loops

05. break and continue statements

06. Sum of natural numbers

07. Factorial using each loop type

08. Fibonacci series up to N terms

09. Prime number check

10. Multiplication table printer

Chapter notes

This chapter introduces repetition. The 10 programs will exercise all three loop forms, demonstrate when each is the natural choice, and use break and continue to control the flow inside a loop.

Chapter 5: Pattern Programs

Status: Coming in the next release. **Number of programs:** 10

Planned topics

01. Right-angled triangle of stars

02. Inverted half-pyramid

03. Full pyramid of stars

04. Diamond pattern

05. Floyd's triangle

06. Pascal's triangle

07. Number pyramid

08. Alphabet pyramid

09. Hollow square pattern

10. Sand-glass pattern

Chapter notes

Pattern programs are the classic interview warm-up. The 10 programs in this chapter will use nested loops to print 10 different shapes, building the spatial reasoning you need for harder problems.

Chapter 6: Arrays

Status: Coming in the next release. **Number of programs:** 10

Planned topics

01. One-dimensional array declaration and traversal

02. Reading and printing N elements

03. Sum and average of array elements

04. Finding the maximum and minimum

05. Linear search

06. Reversing an array

07. Bubble sort

08. Selection sort

09. Insertion sort

10. Binary search on a sorted array

Chapter notes

Arrays are the first data structure. The 10 programs will cover the everyday array operations (read, print, sum, max, reverse) and the three classic sorting algorithms plus binary search.

Chapter 7: Strings

Status: Coming in the next release. **Number of programs:** 10

Planned topics

01. Reading and printing a string

02. Finding the length of a string

03. Copying one string into another

04. Concatenating two strings

05. Reversing a string

06. Checking for a palindrome

07. Counting vowels, consonants and digits

08. Converting case (upper to lower and back)

09. Comparing two strings

10. String library functions (strlen, strcpy, strcat, strcmp)

Chapter notes

Strings are character arrays with a terminating null byte. The 10 programs will exercise the everyday string operations, both hand-rolled (to learn how strings work) and via the library.

Chapter 8: Functions and Recursion

Status: Coming in the next release. **Number of programs:** 10

Planned topics

01. Function declaration, definition and call

02. Functions with parameters and return values

03. Call by value vs call by reference

04. Function prototypes

05. Variable scope (local, global, block)

06. Recursion: factorial

07. Recursion: Fibonacci

08. Recursion: sum of digits

09. Recursion: power function

10. Recursion: tower of Hanoi

Chapter notes

Functions break a program into reusable pieces; recursion is the elegant way to solve self-similar problems. The 10 programs cover the function-call mechanics and five classic recursion problems.

Chapter 9: Pointers and Structures

Status: Coming in the next release. **Number of programs:** 10

Planned topics

01. Pointer declaration and the address-of operator

02. Dereferencing a pointer

03. Pointers and arrays

04. Pointer arithmetic

05. Pointers as function parameters (call by reference)

06. Pointers to pointers

07. Structure declaration and use

08. Array of structures

09. Structures and pointers (-> operator)

10. typedef and enum

Chapter notes

Pointers are what make C unique. The 10 programs build the pointer concept from first principles (address, dereference, arithmetic) and then combine pointers with structures to model real-world records.

Chapter 10: File Handling and Mini Projects

Status: Coming in the next release. **Number of programs:** 10

Planned topics

01. Opening and closing a file (fopen, fclose)

02. Writing characters and strings to a file

03. Reading characters and lines from a file

04. Formatted file I/O (fprintf, fscanf)

05. Binary file I/O (fwrite, fread)

06. File pointer positioning (fseek, ftell, rewind)

07. Mini project: phonebook (add, search, list, delete)

08. Mini project: student marksheet (file-backed)

09. Mini project: simple library manager

10. Mini project: billing system with persistent records

Chapter notes

The final chapter brings together everything: I/O, control flow, arrays, strings, functions, pointers and structures, plus the file functions that let data survive between runs. The 4 mini projects at the end are portfolio-grade and ready to put on GitHub.

Conclusion

A full conclusion will be written once all ten chapters are complete. It will summarise the concepts covered, list the 100 programs by chapter, and suggest next steps — including data structures, embedded C, and the transition to C++ — for learners who finish this collection and want to go further.

Current status: Chapter 1 (10 programs) is fully documented and tested. Chapters 2 through 10 will be released one chapter at a time, with each chapter following the same fourteen-part program layout used in Chapter 1.

Author: Sasidharan S. · **Project:** CodeVault — Programming Practice Library · **Edition:** 2026